Programming Fundamentals | Assignment | Spring 2026
BS-IT-F25 | Morning & Afternoon

Programming Fundamentals
ASSIGNMENT

Spring 2026 — BS-IT-F25 Morning & Afternoon

Submission Deadline: Wednesday, March 25, 2026 (12:00 PM)

Instructions

- This is an individual assignment. Absolutely NO collaboration is allowed. Any traces of plagiarism/cheating or usage of AI tools will result in an "F" grade in this course.
- Do NOT copy even a single line of code from any other person, book, internet, or any other source.
- This assignment needs to be submitted in Soft form. The Submission Procedure is given below.
- **Late submissions will NOT be accepted, whatever your excuse may be.**

Submission Procedure

i. Make .CPP files for the 17 tasks. Name each file: TASK1.cpp, TASK2.cpp ... TASK16.cpp. Include your Name, Roll Number, and Section in comments at the top of each CPP file.

ii. Create an empty folder. Place ONLY the .CPP files inside. Do NOT include any other files — your submission will NOT be graded otherwise.

iii. Compress the folder as a .ZIP file. Name it exactly as:

```
AI-ASS-02-BAIF25M123   (Morning section)
IT-ASS-02-BITF25A456   (Afternoon section)
```

Replace the highlighted part with your complete Roll Number.

iv. Carefully check your .ZIP file to ensure all instructions have been followed.

v. Submit the single .ZIP file on Google Classroom. Make sure you press the SUBMIT button after uploading.

Students Without Laptops:

- Write your Name, Roll Number, and Section at the top of each page.
- Scan your handwritten assignment in PDF format using CamScanner, Adobe Scan, or similar app.
- Rename the PDF using the same format provided above and submit via Google Classroom.

⚠ **Note:** If you do not follow the above submission procedure, your assignment will NOT be graded and you will be awarded ZERO marks.

TASK # 1 | Sum of Squares

Write an algorithm that calculates the sum of squares for a given range.

Requirements

- **User Input:** Ask for a positive integer n.
- **Logic:** The program must print the sum of the squares of all the positive integers strictly smaller than n.

Sample Output

```
Enter a positive integer n: 5


Sum of squares (1² + 2² + 3² + 4²) = 30
```

TASK # 2 | Fibonacci Sequence Generator

Write an algorithm to generate the Fibonacci sequence.

Requirements

- **User Input:** Ask the user for a specific number n.
- **Logic:** Generate and display the Fibonacci sequence up to the limit of n.

Sample Output

```
Enter limit n: 50


Fibonacci sequence up to 50:
0  1  1  2  3  5  8  13  21  34
```

TASK # 3 | Arithmetic Sequence Generator

Write a program that builds a custom arithmetic sequence based on user-defined parameters.

Requirements

- **User Input:** Take the first term (a), common difference (d), and the number of terms (n) from the user.
- **Logic:** Calculate and display the first n terms of the sequence based on the difference.

Sample Output

```
Enter first term (a): 40
Enter common difference (d): 3
Enter number of terms (n): 6


Sequence: 40, 43, 46, 49, 52, 55
```

TASK # 4 | Student Grade Calculator

Write a C++ program that takes input for the marks of 5 students in 3 subjects (Math, Science, English), calculates individual and class-wide statistics, and displays a full summary.

Requirements

- **User Input:** Take the name and marks of 5 students across 3 subjects: Math, Science, and English.
- **Per Student:** Calculate each student's Total marks and Average marks.
- **Class Summary:** Calculate and display the class averages for Math, Science, and English separately.
- **Output:** Display each student's name, total, average, and the class averages for each subject.

Sample Output

```
Enter name and marks for Student 1:
  Name: Ali
  Math: 85   Science: 90   English: 78
```

```
Enter name and marks for Student 2:
  Name: Sara
  Math: 76   Science: 82   English: 91

  ... (5 students total)

--------- Results ---------
Ali    | Total: 253 | Avg: 84.33
Sara   | Total: 249 | Avg: 83.00
...

Class Avg -> Math: 80.20  Science: 85.40  English: 83.60
```

TASK # 5 | Rainfall Statistics

Write a program that uses nested loops to collect monthly rainfall data over multiple years and calculate statistics.

Requirements

- **User Input:** First ask for the number of years. Then for each year, collect rainfall (in inches) for each of the 12 months.
- **Output:** After all data is entered, display: the number of months recorded, total inches of rainfall per year, and the average rainfall per month.

Constraints

- Do not accept a number less than 1 for the number of years.
- Do not accept negative numbers for monthly rainfall.

Sample Output

```
Enter number of years: 2


Year 1:
  Month 1 rainfall (inches): 3.2
  Month 2 rainfall (inches): 2.8
  ... (12 months)


Year 2:
```

```
  Month 1 rainfall (inches): 4.1
  ... (12 months)

-------- Statistics --------
Total months recorded : 24
Year 1 total rainfall  : 38.50 inches
Year 2 total rainfall  : 42.10 inches
Average monthly rainfall: 3.36 inches
```

TASK # 6 | Largest and Second Largest Finder

Write an algorithm to determine the two highest values in a custom list without repetition.

Requirements

- **User Input:** Ask the user for the number of inputs n.
- **Constraints:** Your program must ensure that n is greater than or equal to 2. If n is less than 2, keep prompting the user until a valid value is entered.
- **Logic:** Assume all numbers provided are distinct (no repetition). Determine and display the Largest and the Second largest values.

Sample Output

```
Enter the number of inputs (>=2): -3
ERROR: Invalid input!!

Enter the number of inputs again (>=2): 1
ERROR: Invalid input!!

Enter the number of inputs again (>=2): 4
Enter number 1: 6
Enter number 2: 5
Enter number 3: 8
Enter number 4: 3

Largest number is: 8
Second largest number is: 6
```

TASK # 7 | 24-Hour Digital Clock Simulation

Write a program that simulates a 24-hour digital clock. Display every second of a full day from midnight (00:00:00) to one second before the next midnight (23:59:59).

Requirements

- **Formatting:** Display time in HH:MM:SS format with leading zeros (e.g., 05 instead of 5).
- **C++ Specifics:** Use cout.fill('0') and setw(2) from the <iomanip> library to handle leading zeros.

Sample Output

```
00:00:00
00:00:01
00:00:02
...
00:00:59
00:01:00
...
12:30:45
...
23:59:58
23:59:59
```

TASK # 8 | Binary to Decimal Converter

Design an algorithm to parse a binary string into its decimal equivalent.

Requirements

- **User Input:** Take a binary number (containing only 0s and 1s) from the user.
- **Logic:** Print its decimal (base-10) equivalent.

Sample Output

```
Input a binary number: 101011


The equivalent decimal number is: 43
```

TASK # 9 | Base-7 to Decimal Converter

Input a base-7 number, digit-by-digit, and convert it into the equivalent base-10 number.

Requirements

- **User Input:** Digits of the input will be entered one-by-one in order from Least Significant to Most Significant. Valid digits are 0 to 6.
- **Termination:** The user will enter -1 to terminate the input-taking process.

Note: The sample above corresponds to $634_7 = 319_{10}$

Sample Output

```
Enter a Base-7 digit: 4
Enter a Base-7 digit: 3
Enter a Base-7 digit: 6
Enter a Base-7 digit: -1


The equivalent number in Base-10 is: 319
```

TASK # 10 | Palindrome Number Pyramid

Create a program that generates a centered, symmetrical pyramid of numbers based on a user-provided height n. Each row must be a palindrome centered around the number 1.

Requirements

- **User Input:** Ask the user for the pyramid height n.
- **Each Row:** Row i starts by counting down from i to 1 (Left Wing), then counts back up from 2 to i (Right Wing). Skip the right wing if i = 1.
- **Alignment:** Use leading spaces to center each row and produce the pyramid shape.

Sample Output

```
Enter the value of n: 5
```

```
        1
      2 1 2
    3 2 1 2 3
  4 3 2 1 2 3 4
5 4 3 2 1 2 3 4 5
```

TASK # 11 | Decimal to Base-9 Converter

Input a base-10 number and convert it into its base-9 equivalent.

Requirements

- **User Input:** Ask for a decimal number.
- **Logic:** Store its equivalent in base-9 as a single numeric value, and display it on screen.

Note: The sample above corresponds to $953_{10} = 1268_9$

Sample Output

```
Enter a decimal (Base-10) number: 953


The equivalent number in Base-9 is: 1268
```

TASK # 12 | Paired Sequence Generator

Write a C++ program using nested loops to display a cascading number pairing sequence.

Requirements

- **User Input:** Ask for a starting integer and an ending integer.
- **Logic:** Format the pairs exactly as shown in the sample output below.

Sample Output

```
Enter starting integer: 11
```

```
Enter ending integer: 18


(11,11) (11,12) (11,13) (11,14) (11,15) (11,16) (11,17) (11,18)
(12,12) (12,13) (12,14) (12,15) (12,16) (12,17)
(13,13) (13,14) (13,15) (13,16)
(14,14) (14,15)
```

TASK # 13 | Pythagorean Triples Finder

Find and display all Pythagorean triples where the sum of the squares of two sides equals the square of the hypotenuse.

Requirements

- **Logic:** Find all combinations for side1, side2, and hypotenuse.
- **Constraints:** All three values must be integers no larger than 200.
- **Hint:** Use a triple-nested for loop that tries all possibilities.

Sample Output

```
(3, 4, 5)   ->  3² + 4²  = 5²
(5, 12, 13) ->  5² + 12² = 13²
(6, 8, 10)  ->  6² + 8²  = 10²
...
(All triples up to 200 are displayed)
```

TASK # 14 | Square Multiplication Table

Write a C++ program to print a dynamically sized multiplication grid.

Requirements

- **User Input:** Take a positive integer N from the user.
- **Formatting:** Print a multiplication table of size N by N. You must use appropriate format manipulator(s) (like setw()) to align the columns perfectly.

Sample Output

```
Enter N: 5


    1    2    3    4    5
    2    4    6    8   10
    3    6    9   12   15
    4    8   12   16   20
    5   10   15   20   25
```

TASK # 15 | Plus Sign Pattern

Print a staggered geometric pattern based on custom dimensions.

Requirements

- **User Input:** Take height and width from the user.
- **Logic Restriction:** You are ONLY allowed to use the following three output statements in your code: cout << "+"; cout << " "; cout << endl;

Constraints

- Both height and width must be odd numbers.
- Height must be >= 3 and width must be >= 5.
- Keep prompting the user if the input is invalid.

Sample Output

```
Enter height: 5
Enter width: 11


++++++++++
+ + + + + +
++++++++++
+ + + + + +
++++++++++
```

TASK # 16 | Binary Square Pattern

Write a C++ program which prints a diagonal block pattern of 1s and 0s.

Requirements

- **User Input:** Take the size (dimensions) from the user.
- **Logic Restriction:** You are ONLY allowed to use the following three output statements in your code: cout << "0"; cout << "1"; cout << endl;

Constraints

- The size must be an even number >= 2.
- Keep prompting the user if the input is invalid.

Sample Output

```
Enter size: 6


111111
011111
001111
000111
000011
000001
```

TASK # 17 | Concentric Number Square

Write a C++ program that prints a **concentric number square pattern** based on the input size.

Requirements

- **User Input: Take the size *(n)* from the user.**
- **Pattern Logic:**
  - **The outer boundary of the square should contain 1's.**
  - **Moving towards the center, the numbers should increase by 1 in each inner layer.**
  - **The pattern must remain perfectly symmetric horizontally and vertically.**

Constraints

- The size **n must be an odd number ≥ 1**.

➤ Keep prompting the user until a valid value is entered.

Sample Output

```
Enter size: 5


1 1 1 1 1
1 2 2 2 1
1 2 3 2 1
1 2 2 2 1
1 1 1 1 1
```

Sample Output

```
Enter size: 7


1 1 1 1 1 1 1
1 2 2 2 2 2 1
1 2 3 3 3 2 1
1 2 3 4 3 2 1
1 2 3 3 3 2 1
1 2 2 2 2 2 1
1 1 1 1 1 1 1
```

GOOD LUCK!

Start early — write clean code — submit on time.